REPORT ON PROJECT INTRACRANIAL HEMORRHAGE (ICH) SEGMENTATION USING U-NET

1. **Dataset description**

   This dataset contains 82 CT scans from patients with a mean age of 27.8 years, including 36 cases of intracranial hemorrhage. Each scan consists of approximately 30 slices at 5mm thickness, annotated by two radiologists who delineated hemorrhage regions without access to clinical history and reached consensus on all findings. The DICOM files were processed using Siemens syngo software with brain and bone windowing settings, and the resulting segmentation masks were saved as NIfTI files for 75 of the 82 subjects.
   For more information about this data set, please refer to this URL:
   https://physionet.org/content/ct-ich/1.3.1/#files-panel

2. **Task Definition**
   - In this project, a pre-trained U-Net framework is used to perform pixel-level segmentation of intracranial hemorrhage (ICH) regions in CT scan slices. The trained model is expected to generalize to unseen CT scans, enabling faster diagnosis and timely treatment decisions for patients with ICH.
   - U-Net was selected as the backbone architecture due to its proven effectiveness in medical image segmentation tasks, particularly for detecting small irregularly shaped regions such as hemorrhage areas in CT scans.

3. **Data preprocessing**

   This line code **ct_scans_dir, masks_dir = extract_img_label_each_slice()** will perform the following tasks:
   - Loads the CT slices (grayscale images) of the brain-window for each patient in ct_scans/mask folder then saves them to one folder *(data\image and data\label)*
   - There are data for 75 patients. Each patient has multiple CT images/slices *(input data)* and masks *(ground true)* for those slices
   - These masks are 3D matrices with only 0 and 1 values. 0 means background, no hemorrhage, 1 means hemorrhage (raw mask)
   - We then clip values outside the window — anything below -20 HU (Hounsfield Units — the standard measurement scale for CT scan pixel) (0 pixel) becomes 0 (black), anything above 100 HU (255 pixel) becomes 255 (white) because:
     - image format (PNG, JPG) only support 0-255.
     - neural networks train better with normalized inputs
     - exclude irrelevant tissues (bone, air) (HU < -20 and > 100)
   - Similarly, we transform values 0,1 of mask to 0,255 to save them as images

4. **Split the data set**
   - Split the data set into training, validation, and test set at the ratio 70/20/10 and then retrieve corresponding slices

- Call file ***split_data_set.py*** in the terminal to perform the splitting
- The separated set will be stored in different files as below:
  ./data/train_split.csv, ./data/val_split.csv, ./data/test_split.csv
- Result after splitting

```
(dl_project) [tnguye47@c0219 material]$ python split_data_set.py
Train slices:      1698
Validation slices: 818
Test slices:       298
```

5. **Augmentation**
   - As dataset contains **2814** slices which are quite small for training deep learning network, we need to create new training images by applying transformations to existing ones
   - Augmentation will help with:
     - Increasing dataset size
     - Teaching the model to be invariant to the variance of each sample hemorrhage
     - Preventing overfitting: the model can **generalize** patterns to predict on unseen patients
   - This is the hemorrhage rate and non-hemorrhage rate in the training set before augmentation

```
Total slices in training set: 1698
Slices with hemorrhage: 195 (11.5%)
Slices without hemorrhage: 1503 (88.5%)
```

   - The **augmentation will be done only on the hemorrhage slices** because the number of slices without hemorrhage is far greater than the number of slices with hemorrhage
   - Three augmented versions are generated per slice. Total number of slides after augmentation is 2283, so the **number of added slides with hemorrhage** is 2283 – 1698 = 585

```
After augmentation:      2283
```

   - In this project, I used the below augmentations with corresponding reasons

Augmentation	Reason
Horizontal flip	Brain is roughly symmetric
Small rotation (±10°)	Patient positioning varies
Brightness/contrast	Scanner variations
Zoom/crop	Different patient sizes

6. **Training:**
   - Call file ***train_net.py*** in the terminal to perform the training
   - The whole data set will be normalized into values 0 and 1 which helps the gradient converge faster.
   - Model name: **resnet34**
   - Number of batches for each batch for each epoch: **16**
   - Learning rates: **1e-5**
   - Number of training epochs: **200**

- A pixel-level class imbalance check is performed to compute **pos_weight** — the ratio of background to hemorrhage pixels capped at 5.0 — which is used to penalize missed hemorrhage detections more heavily during training.

```
# Check imbalance at pixel level
# sum over channel+H+W → one value per slice
# result = [1000, 0, 500]   # shape: (3,) — one value per slice, so slice
total_pixels = y_train.sum(axis=(1,2,3))
pos_pixels = total_pixels.sum()           # total hemorrhage pixels
# y_train.size = 3 × 1 × 512 × 512 = 786,432   # total pixels
neg_pixels = y_train.size - pos_pixels    # total background pixels
# pos_weight = neg_pixels / pos_pixels
pos_weight = min(neg_pixels / pos_pixels, 5.0)
print(f"Pixel-level pos_weight: {pos_weight:.1f}")  # expect 50-200x
```

- Optimization function:

```
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

- Loss function for training set

```
loss = torch.nn.functional.binary_cross_entropy_with_logits(preds, masks, pos_weight=pw, reduction='mean')
```

- Loss function for validation set

```
val_loss = torch.nn.functional.binary_cross_entropy_with_logits(pred_val, masks, pos_weight=pw, reduction='mean')
```

- Accuracy metric for validation dataset
  - Since our masks are heavily imbalanced, most pixels are background (0). A lazy model that predicts all zeros could get 95% accuracy but be completely useless because it missed completely 5% of hemorrhage pixels so we can't use normal accuracy metric for validation performance.
  - Dice score specifically measures overlap between predictive hemorrhage and actual hemorrhage, so it penalizes that lazy behavior.
  - A high dice score means highly overlap between prediction and ground truth. A perfect prediction would give Dice = 1.0, and no overlap at all would give Dice = 0.0
  - The Dice score formula is:

$$Dice = \frac{2 \times | A \cap B |}{| A | + | B |}$$

    Where:
    - **A** = predicted binary mask
    - **B** = ground truth mask
    - **|A ∩ B|** = number of overlapping pixels (intersection)
    - **|A|** = number of predicted positive pixels
    - **|B|** = number of ground truth positive pixels
- Early stopping is evaluated based on validation Dice score as this metric determines directly how well the model is detecting hemorrhage.

- This method will terminate training if there is no improvement observed after running **30 epochs starting from the best epoch**, reducing training time and unnecessary computation.

- Accuracy metric for test dataset
  - Dice score
  - Sensitivity: clinically important, reflects how well the model catches the actual hemorrhage pixels
  - The formula for sensitivity is:

$$Sensitivity = \frac{TP}{TP + FN}$$

    Where:
    - TP (True Positive) — hemorrhage pixels correctly predicted as hemorrhage
    - FN (False Negative) — hemorrhage pixels incorrectly predicted as background (missed hemorrhage)
  - Low sensitivity means the model is missing real hemorrhages

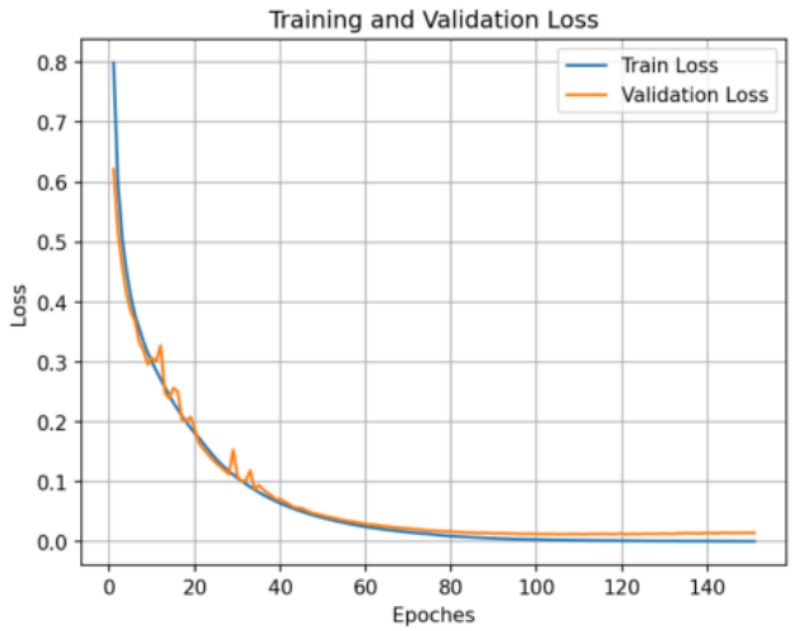
7. **Training results**

- Early stopping results

```
Total val slices loaded: 818
Positive val slices loaded: 88
Negative val slices loaded: 730
Total positive pixels in val: 138352
Pixel-level pos_weight: 5.0
Early stopping at epoch 151. Best dice score =0.341. Best epoch=121
Training took 155.68 minutes
```

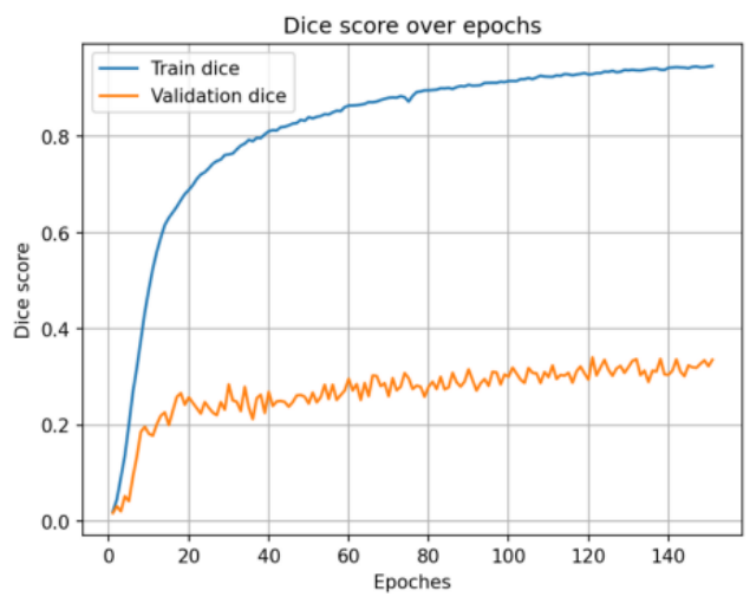
➔ Best dice score we can get is 0.341 at epoch 151, after that epoch, the model performance is not improved anymore

- Training and validation loss



- Both the training curve and validation curve for the loss is converged to nearly zero. There is no gap between these two curves. Plots of both training and validation look similar because most of the pixels are background and the model predicts the background correctly on both sets.

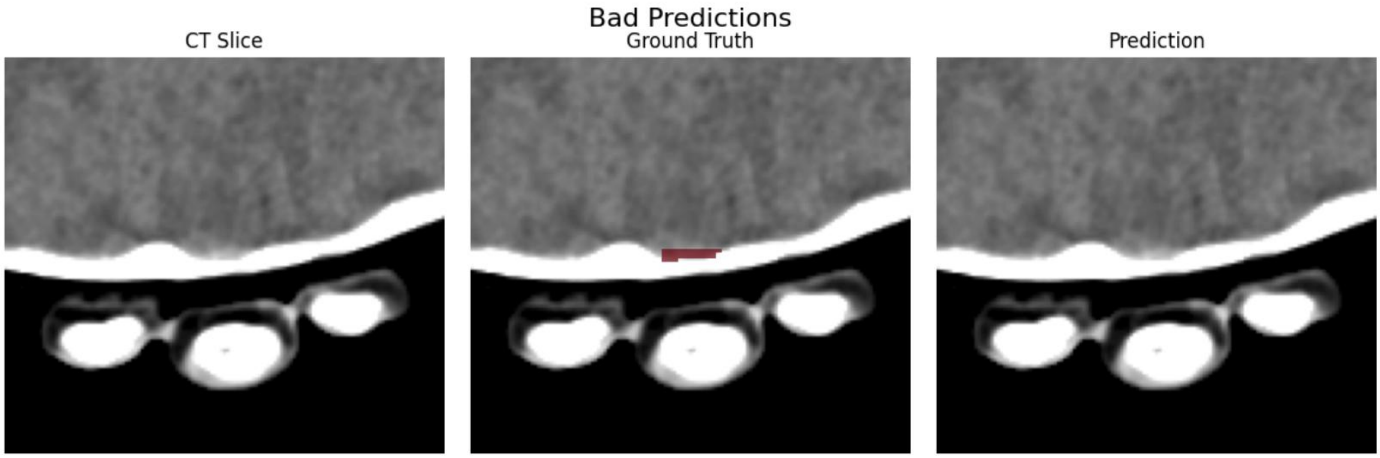
- Dice score on the training and validation set

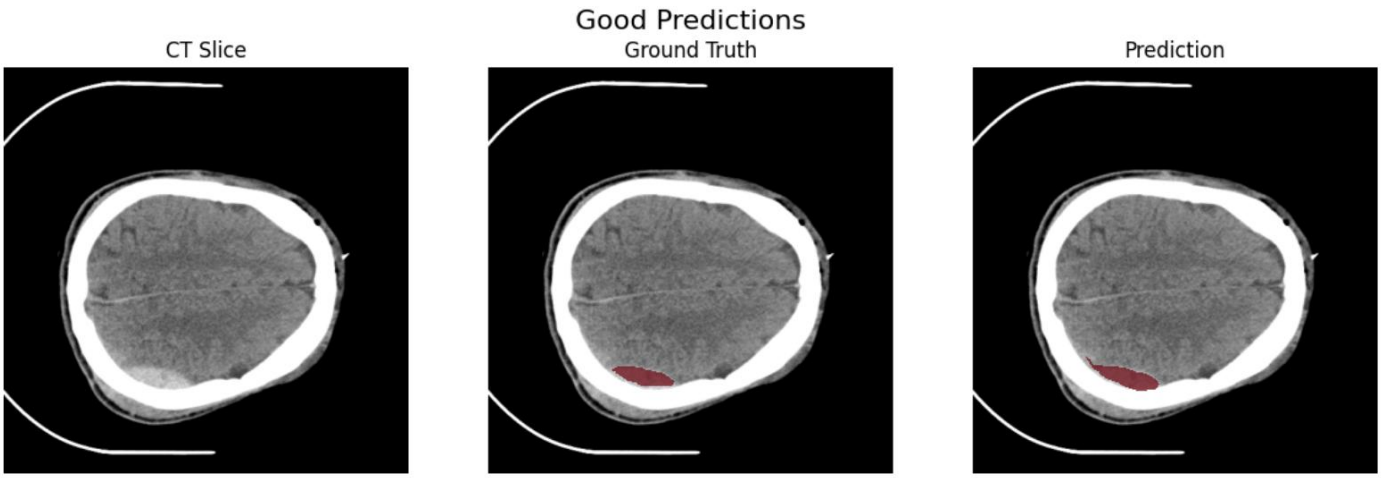


- The dice score is computed only on positive slices. It measures how well the model finds the small hemorrhage regions. The model has memorized the exact hemorrhage shapes in the 195 training slices (TrainDice = 0.94) but fails to generalize to unseen hemorrhage shapes in the validation set (ValDice = 0.341).

- The dice score reaches the peak at epoch 121 and then stops improving and goes stable in range of 0.001–0.003, suggesting the model has reached its capacity limit given the current dataset size and architecture, with no further meaningful improvement despite continued training.

8. **Testing process**
   - Run file ***train_test_ich.ipynb*** to perform the testing
   - The best model which is saved at epoch 121 is loaded to perform the testing.
   - Dice score is 0.321 and sensitivity is 0.319
     - A Dice score of 0.321 means that on average, only about **32% of the predicted and actual hemorrhage pixels overlap**, indicating the model can locate hemorrhage regions but struggles with precise boundary delineation. This is a moderate result for medical image segmentation, where scores above 0.70 are typically considered clinically useful.
     - Sensitivity measures how well the model detects **actual hemorrhage pixels** — out of all real hemorrhage pixels in the test set, the model correctly identified only **31.9%** of them. This means the model misses approximately 69% of hemorrhage pixels, which is a significant concern for clinical use since missed hemorrhages (false negatives) are more dangerous than false alarms.
   - Image illustration for bad and good predictions





9. **Refinement / Improvement Strategies**
   - Because the number of hemorrhage pixels is very small compared to background pixels in a CT slice (e.g. 4 hemorrhage vs 1000 background), BCE (binary cross entropy) loss is dominated by the majority class. This means the model can "cheat" by predicting everything as background and still achieve a very small loss, since the few missed hemorrhage pixels contribute negligibly to the total.
   - Furthermore, BCE evaluates each pixel **independently** with no awareness of spatial relationships — it only asks "is this pixel right or wrong?" but never asks "are the correct pixels in the right location relative to each other?" This means even if the model predicts the right **number** of hemorrhage pixels but in the **wrong location**, BCE loss would still be low. Dice loss addresses this by measuring the **overlap** between predicted and actual hemorrhage regions — if the predicted pixels are in the wrong location, the intersection is small, Dice score is also small. That's why, we should include Dice score in computing loss for **positive slices only** to directly penalize spatial misalignment.

     Combined Loss=α×BCE Loss+(1−α)×Dice Loss

     - **α:** weighting factor or balancing coefficient: it controls the relative contribution of each loss component.
   - Below is a list of optimized points that I used to improve the dice scores for the validation set
     - Combing binary cross-entropy and dice loss
     - Using AdamW optimization function instead of Adam function to penalize large weights, forcing models to learn more general features.
     - Applying CosineAnnealingLR schedule to gradually reduce learning rates following a cosine curve shape over the training
     - Adding gradient clipping: to prevent the exploding gradients issue. Gradient clipping sets a maximum allowed gradient magnitude
       - If gradient magnitude > 1.0 → scale it down to 1.0

- If gradient magnitude < 1.0 → leave it unchanged

- Using **Optuna** library to perform hyper-parameter tuning for learning rate, pos_weight and alpha (α)

- Run file ***train_net_hyper_finetune.py*** and below is the result

```
Training took 7.59 hours
 Trial  Best Dice       LR  Pos Weight  Alpha
     0     0.4441 1.56e-03        8.72  0.627
     3     0.4279 8.26e-04        4.18  0.516
     6     0.4231 1.07e-03        5.95  0.488
     5     0.4121 4.22e-04        7.19  0.659
     2     0.4105 1.70e-03        2.73  0.588
     1     0.4028 4.63e-04        4.42  0.476
     8     0.4015 4.95e-04        2.88  0.326
     7     0.3982 5.46e-04        7.69  0.391
     9     0.3940 4.50e-04        3.09  0.603
     4     0.3884 5.57e-04        5.16  0.464
```

```
Best Trial: 0
Best Dice:  0.4441
Best Params:
  lr: 0.0015610793334561123
  pos_weight: 8.719169548163414
  alpha: 0.626570318177089
```

- Key observations
  - Learning rates
    Top 3 trials: lr = 1.56e-03, 8.26e-04, 1.07e-03 ← all above 8e-04
    Bottom 3 trials: lr = 4.50e-04, 5.46e-04, 5.57e-04 ← all below 6e-04
    ➔ This confirms the optimal learning rate is in the higher range
  - Pos_weight
    Top trials: 8.72, 4.18, 5.95
    → varies widely, and has no clear pattern.
  - Alpha
    Top trials: 0.627, 0.516, 0.488
    ➔ Slightly more weight on BCE then Dice works better
  - Best Dice score on the validation set is improved from **0.341 to 0.444**, which is a little bit better than before.
  - The low dice score is understandable on a highly imbalanced data set, with **11.5%** of slices having hemorrhage regions and **88.5%** of slices without hemorrhage regions. To address this, we can consider increasing the number of augmentation data from 3 to 5 and observe the result again.